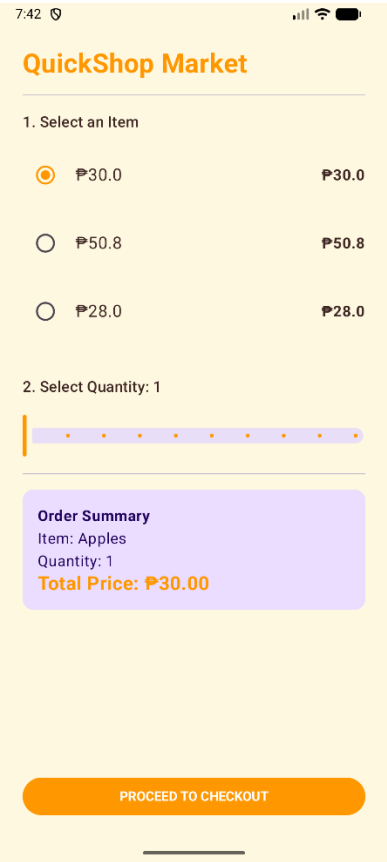
Sean Mikael Bernard Cid

Application Description

This application lets you select any items and lets you select the quantity you want to buy and it shows you the prices of the item and what would be the total of price depending on the quantity of the item



```
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContent {
            CidAct3Theme {
                Scaffold(modifier = Modifier.fillMaxSize()) { innerPadding ->
                    StoreApp(
                        modifier = Modifier.padding( paddingValues = innerPadding)
                    )
                }
            }
        }
    }
}

2 Usages
@Composable
fun StoreApp(modifier: Modifier = Modifier) {
    // Selection state
    var selectedItem by remember { mutableStateOf( value = "Apples") }
    // Sliding component for quantity
    var quantity by remember { mutableFloatStateOf( value = 1f) }
    // State for Alert Dialog
    var showDialog by remember { mutableStateOf( value = false) }

    val items = listOf(
        StoreItemData( name = "Apples",  price = 30.0),
        StoreItemData( name = "Bananas",  price = 50.80),
```

```
    StoreItemData( name = "Bananas",  price = 50.80),
    StoreItemData( name = "Cherries",  price = 28.00)
)

Column(
    modifier = modifier
        .fillMaxSize()
        .padding( all = 24.dp),
    verticalArrangement = Arrangement.spacedBy(16.dp)
) {
    Text(
        text = "QuickShop Market",
        fontSize = 28.sp,
        fontWeight = FontWeight.Bold,
        color = MaterialTheme.colorScheme.primary
    )

    HorizontalDivider()

    Text(text = "1. Select an Item", style = MaterialTheme.typography.titleMedium)
    items.forEach { item ->
        Row(
            modifier = Modifier
                .fillMaxWidth()
                .clickable { selectedItem = item.name }
                .padding(vertical = 4.dp),
            verticalAlignment = Alignment.CenterVertically
        ) {
            RadioButton(
                selected = (selectedItem == item.name),
                onClick = { selectedItem = item.name }
```

```
                selected = (selectedItem == item.name),
                onClick = { selectedItem = item.name }
            )
            Text(text = "₱${item.price}", fontSize = 18.sp, modifier = Modifier.padding(start = 8.dp))
            Spacer(modifier = Modifier.weight(1f))
            Text(text = "₱${item.price}", fontWeight = FontWeight.Bold)
        }
    }

    Spacer(modifier = Modifier.height(8.dp))

    Text(text = "2. Select Quantity: ${quantity.toInt()}", style = MaterialTheme.typography.titleMedium)
    Slider(
        value = quantity,
        onValueChange = { quantity = it },
        valueRange = 1f ≤ .. ≤ 10f,
        steps = 8
    )

    HorizontalDivider()

    val basePrice = items.find { it.name == selectedItem }?.price ?: 0.0
    val finalTotal = basePrice * quantity.toInt()
    val formattedTotal = String.format("%.2f", finalTotal)

    Card(
        modifier = Modifier.fillMaxWidth(),
        colors = CardDefaults.cardColors(containerColor = MaterialTheme.colorScheme.primaryContainer)
```

```
        Card(
            modifier = Modifier.fillMaxWidth(),
            colors = CardDefaults.cardColors(containerColor = MaterialTheme.colorScheme.primaryCont
        ) {
            Column(modifier = Modifier.padding( all = 16.dp)) {
                Text(text = "Order Summary", fontWeight = FontWeight.Bold)
                Text(text = "Item: $selectedItem")
                Text(text = "Quantity: ${quantity.toInt()}")
                Text(
                    text = "Total Price: ₱$formattedTotal",
                    fontSize = 20.sp,
                    fontWeight = FontWeight.Bold,
                    color = MaterialTheme.colorScheme.primary
                )
            }
        }

        Spacer(modifier = Modifier.weight(1f))

        Button(
            onClick = { showDialog = true },
            modifier = Modifier.fillMaxWidth()
        ) {
            Text("PROCEED TO CHECKOUT", fontWeight = FontWeight.Bold)
        }

        if (showDialog) {
            AlertDialog(
                onDismissRequest = { showDialog = false },
                title = { Text(text = "Confirm Order") },
```

```
            AlertDialog(
                onDismissRequest = { showDialog = false },
                title = { Text(text = "Confirm Order") },
                text = { Text(text = "Do you want to complete your purchase of ${quantity.toInt()} $selectedItem for ₱$formattedTotal?") },
                confirmButton = {
                    TextButton(onClick = { showDialog = false }) {
                        Text("Confirm")
                    }
                },
                dismissButton = {
                    TextButton(onClick = { showDialog = false }) {
                        Text("Cancel")
                    }
                }
            )
        }
    }
}

3 Usages
data class StoreItemData(val name: String, val price: Double)

@Preview(showBackground = true)
@Composable
fun StoreAppPreview() {
    CidAct3Theme {
        StoreApp()
    }
```

Radio Button –

```
val items = listOf(
    StoreItemData("Apples", 30.0),
    StoreItemData("Bananas", 50.80),
    StoreItemData("Cherries", 28.00)
)

RadioButton(
        selected = (selectedItem == item.name),
        onClick = { selectedItem = item.name }
    )
    Text(text = "₱${item.price}", fontSize = 18.sp, modifier = Modifier.padding(start = 8.dp))
    Spacer(modifier = Modifier.weight(1f))
    Text(text = "₱${item.price}", fontWeight = FontWeight.Bold)
}
```

For each item that was on the list it creates a radio button that has the name and price that was on the list

Slider

```
Text(text = "2. Select Quantity: ${quantity.toInt()}", style = MaterialTheme.typography.titleMedium)
Slider(
    value = quantity,
    onValueChange = { quantity = it },
    valueRange = 1f..10f,
```

```
    steps = 8
)
```

the Slider component allows users to pick a value between 1f and 10f. The steps = 8 attribute is used to create discrete "snapping" points so the user can only select whole numbers (1, 2, 3...).

Alert Dialog

this component is conditionally rendered (if (showDialog) { ... }). It is triggered when the "PROCEED TO CHECKOUT" button is clicked. It uses a title for the heading and a text block to show the final order summary (item name, quantity, and total price). The confirmButton and dismissButton both reset showDialog to false to close the popup